



Mock examination opportunity 3 memo(past paper)

Computer science (University of Pretoria)



Scan to open on Studocu



UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

*Faculty of EBIT/ Fakulteit van IBIT
Department of Computer Science/ Department van Rekenaarwetenskap
Program Design: Introduction / Programmeringontwerp: Inleiding
COS 110*

*Mock Exam Opportunity 3
2017/11/02*

Instructions

1. This is a mock question paper. The provided questions will not feature in the EO3.
2. The purpose of this question paper is to help you prepare for the upcoming EO3.
3. The memo to the question paper will not be provided.

Marks:

Question	Points	Score
Queues and Stacks / <i>Toue en Stapels</i>	28	
Recursion / <i>Rekursie</i>	14	
Linked lists and Recursion / <i>Geskakelde lyste en Rekursie</i>	32	
Total:	74	

Question 1 Queues and Stacks / *Toue en Stapels* (28 marks)

- 1.1 True or false: a queue is both a first in, first out, as well as a last in, last out data structure. (2)
Motive your answer. /

*Waar of vals: 'n tou is beide 'n eerste in, eerste uit, sowel as 'n laaste in, laaste uit datastruktuur.
Motiveer jou antwoord.*

- 1.2 Name one difference between a stack and a queue. / (2)
Noem een verskil tussen 'n stapel en 'n tou.

- 1.3 Will it make sense to implement a circular stack? Give a reason for your answer. / (2)
Sal dit sin maak om 'n sirkulêre stapel te implementeer? Gee 'n rede vir jou antwoord.

- 1.4 Will it make sense to implement a circular queue? Give a reason for your answer. / (2)
Sal dit sin maak om 'n sirkulêre tou te implementeer? Gee 'n rede vir jou antwoord.

1.5 For each of the following practical situations, indicate whether or not a queue would be an appropriate data structure, and provide an explanation for your answer. /

Vir elk van die volgende praktiese situasies, dui aan of 'n tou 'n gepaste datastruktuur sou wees of nie, en verduidelik jou antwoord.

- a) A video file is downloaded from a web server. The individual frames are downloaded in play order and are stored in a temporary buffer, until the entire video is downloaded and ready to be played. The temporary buffer is implemented as a queue. / (2)
- 'n Video-lêer word vanaf 'n web bediener afgelaai. Die individuele rame word in speelvolgorde afgelaai en in 'n tydelike buffer gestoor totdat die hele video afgelaai is en reg is om gespeel te word. Die buffer is as 'n tou geïmplementeer.*

- b) A string containing several nested pairs of brackets is checked for well-formedness. A string is well-formed if each opening bracket has a closing bracket of the same type. Furthermore, while nesting is allowed, each inner pair of brackets should be closed before its outer pair. For example, `{ }` is well-formed, but `{ }` is not. The string is scanned from left to right. When an opening bracket is encountered, it is enqueued to a temporary queue. When a closing bracket is encountered, an opening bracket is dequeued from the buffer, and its type is checked against the closing bracket. If both brackets' types match, the string is well-formed. / (2)
- A string wat verskeie geneste hakies bevat word getoets om te bepaal of dit goed gevormd is. 'n String is goed gevorm as elke oop hakie 'n toe hakie van dieselfde tipe het. Verdermeer, alhoewel hakies genes kan wees, moet elke binneste paar toegemaak word voor sy buitenste paar. Byvoorbeeld, `{ }` is goed-gevormd, maar `{ }` is nie. Die string word van links na regs gelees. Elke oop hakie word in 'n tydelike tou geplaas. Wanneer 'n toe hakie gevind word, word 'n oop hakie vanuit die tou gehaal, en die oop hakie se tipe word dan vergelyk met die toe hakie s'n. As albei hakies se tipe dieselfde is, is die string goed gevorm.*

1.6 The STL provides similar classes called **stack** and **queue**, both of which have functions called **push** and **pop**. Answer the following questions:/

*Die STL verskaf soortgelyke klasse genaamd **stack** en **queue**, waarvan beide funksies genaamd **push** en **pop** het. Beantwoord die volgende vrae:*

- a) Does **push** perform a different action in the **stack** class than in the **queue** class? Explain your answer. / (2)
- Voer **push** 'n ander aksie in die **stack** klas uit as in die **queue** klas? Verduidelik jou antwoord.*

- b) Does **pop** perform a different action in the **stack** class than in the **queue** class? Explain your answer. / (2)
*Voer **pop** 'n ander aksie in die **stack** klas uit as in die **queue** klas? Verduidelik jou antwoord.*

- 1.7 Suppose you are tasked with implementing a reverse queue in which elements are enqueued at the front and dequeued at the rear. The implementation is dynamic, and you are using the following **struct** for the nodes that are containing the elements: /
*Veronderstel jy moet 'n agterstevoor tou implementeer, waar elemente aan die voorkant bygevoeg word en van die agterkant verwyder word. Die implementasie is dinamies, en jy gebruik die volgende **struct** vir die nodusse wat die elemente bevat:*

```
struct Node {
    char element;
    Node *next;
};
```

Suppose your queue class has **Node** pointers called **front** and **rear**, which point to the front and rear nodes in the queue. /
*Veronderstel jou tou het **Node** wysers genaamd **front** en **rear** wat na die voorste en agterste nodusse in die tou wys.*

- a) Implement **enqueue** with the following signature: / (3)
*Implementeer **enqueue** met die volgende verklaring:*
void RQueue::enqueue(**char** el);

- b) Implement **dequeue** with the following signature (the function should return the dequeued value): /
*Implementeer **dequeue** met die volgende prototipe (die funksie moet die verwyderde element terugstuur):*

```
char RQueue::dequeue();
```

- c) Does the different (reverse) implementation affect the behaviour of the queue from the client's perspective? Explain your answer. / *Veroorsaak die ander (agterstevoor) implementasie die gedrag van die tou vanuit die klient se perspektief? Verduidelik jou antwoord.*

1.8 The STL **stack** class is a container adapter that uses another container internally. It provides a simple stack-like interface by using the functions **push_back**, **pop_back** and **back** on its internal container. The adapter can only be used on certain containers that provide those functions. You are tasked with creating a new container adapter called **StackQueue**, which uses an STL **queue** as an internal container. The class has the following simple definition: /

*Die STL stapel (**stack**) klas is 'n houer-aanpasser wat 'n ander houer intern gebruik. Dit verskaf 'n eenvoudige stapel-agtige gedrag deur die funksies **push_back**, **pop_back** en **back** op sy interne houer te roep. Die aanpasser kan slegs op sommige houters gebruik word wat wel daardie funksies verskaf. Jy word gevra om 'n nuwe houer-aanpasser genaamd **StackQueue** te skep wat 'n STL tou (**queue**) as 'n interne houer gebruik. Die klas het die volgende eenvoudige definisie:*

```
template <class T>
class QueueStack {
    queue<T, deque<T> > q;
public:
    bool empty() { return q.empty(); }
    void push(T);
    T pop();
};
```

Note that **pop**'s return type is **T**; it should return the element that is begin popped from the stack.
*/ Let wel dat **pop** se terugkeertipe **T** is; dit moet die element wat van die stapel verwyder word terugstuur.*

Assume that the **push** function is implemented as follows: /

*Aanvaar dat die **push** funksie soos volg geïmplementeer is:*

```
template <class T> void QueueStack<T>::push(T el){
    q.push(el);
```

- a) Provide an implementation for the **pop** function. You may use a temporary **queue** object if needed. / (3)

*Verskaf die implementasie vir die **pop** funksie. Jy mag 'n tydelike **queue** objek gebruik waar nodig.*

```
template <class T> T QueueStack<T>::pop();
```

Question 2 Recursion / *Rekursie* (14 marks)

2.1 Why would a programmer most likely not use recursion to implement a string library, but rather use iteration? / *Waarom sal 'n programmeerder waarskynlik nie rekursie gebruik om 'n string biblioteek te implementeer nie, maar eerder van iterasie gebruik maak?* (2)

2.2 Each of the following recursive functions has an error in the code. For each of these functions, write down what the error is, as well as how the error can be corrected. / *Elkeen van die volgende rekursiewe funksies het 'n fout in die kode. Skryf vir elkeen van die funksies neer wat die fout is en hoe die fout reggemaak kan word.* (2)

a) `void foo(int i) {
 if (i < 1) return;
 cout << i << endl;
 foo(i);
}` (2)

b) `void printZeroToN(int n) {
 if (n < 0) return;
 cout << n- << endl;
 return printZeroToN(n);
}` (2)

2.3 This question refers to the following algebraic equation: /
Hierdie vraag verwys na die volgende algebraïese vergelyking:

$$y = 1 - x + \frac{x^2}{2} - \frac{x^3}{6} + \frac{x^4}{24} + \cdots + (-1)^n \left[\frac{x^n}{n!} \right]$$

Assume that you are going to solve the algebraic equation with recursion. /
Aanvaar that jy die algebraïese funksie met rekursie gaan oplos.

a) Write down the base case for the recursive function. / (1)
Skryf die basisgeval vir die rekursiewe funksie neer.

b) Write down the recursive case for the recursive function. / (1)
Skryf die rekursiewe geval vir die rekursiewe funksie neer.

- c) Write down a recursive function to solve the algebraic equation. Take note that the factorial in the equation should also be solved with recursion and its recursive function should be written down. The function signature of the function solving the algebraic equation is as follows: / (6)

Skryf 'n rekursiewe funksie neer om die algebraïese vergelyking op te los. Neem kennis dat die faktoriaal in die vergelyking ook met rekursie opgelos moet word, en dat die rekursiewe funksie vir die faktoriaal neergeskryf moet word. Die funksieverklaring van die funksie wat die algebraïese funksie oplos is soos volg:

```
double calculateEquation(int x, int n);
```

Question 3 Linked lists and Recursion / *Geskakelde lyste en Rekursie* (32 marks)

Consider the following linked node **struct** and dynamic queue class: /
Beskou die volgende geskakelde nodus en dinamiese tou klas:

```

template <class T>
struct Node {
    T value;
    Node<T> *next;
};

template <class T>
class Dynque
{
    protected:
        Node<T> *front;
        Node<T> *rear;
        int numItems;
    public:
        Dynque();
        Dynque(Dynque&);
        virtual ~Dynque();
        virtual void enqueue(T); // add an element to rear
        T dequeue(); // remove an element from front
        bool isEmpty();
        int getNumItems();
        void clear(); // remove all elements
        void reverse(); // reverse order of elements
};

```

- 3.1 Implement the copy constructor of the **Dynque** class. / (4)
*Implementeer die kopie konstrueerder van die **Dynque** klas.*

3.2 Implement `clear()` recursively. /
Implementeer `clear()` rekursief.

(3)

3.3 Implement `reverse()` iteratively. /
Implementeer `reverse()` iteratief.

(5)

- 3.4 Implement **reverse()** **recursively** without using any external/additional data structures. / (4)
*Implementeer **reverse()** **rekursief** sonder enige eksterne/addisionele datastrukture.*

- 3.5 Consider the following sorted dynamic queue class: (7)
Beskou die volgende gesorteerde dinamiese tou klas:

```
template <class T>
class SortedDynque : public Dynque<T>
{
    public:
        SortedDynque();
        SortedDynque(SortedDynque&);
        ~SortedDynque();
        Node<T> * begin() { return front; }
        // search for a value, return true if found:
        bool binarySearch(Node<T> * start, Node<T> * end, int size, T val);
};
```

Assume **SortedDynque** stores elements in ascending order, and inherits all variables and functions from the previously defined **Dynque** class. /

*Aanvaar **SortedDynque** stoor elemente in dalende order, en oorerf al die veranderlikes en funksies van die vorige gedefiniëerde **Dynque** klas.*

Implement the **binarySearch(Node<T> * start, Node<T> * end, int size, T val)** function recursively. The function uses binary search to search the queue for the presence of value **val** of type **T**. Parameters are: / *Implementeer die **binarySearch(Node<T> * start, Node<T> * end, int size, T val)** funksie rekursief. Die funksie gebruik 'n binêre soektog om die tou te deursoek vir 'n spesifieke waarde **val** of type **T**. Parameters is:*

- **Node<T> * start** - start address of the search / *begin adres van die soektog*

- `Node<T> * end` - end address of the search (when end reached, stop search) / *adres waar soektog ophou (wanneer die einde bereik is, stop die soektog)*
- `int size` - size of the list to be searched / *grootte van die lys wat deursoek moet word*
- `T val` - value you are searching for / *waarde waarvoor gesoek word*

If the value is found, the function returns `true`. Otherwise, the function returns `false`. Assume the initial function call to search the entire queue is: /

*As die waarde gevind is, moet die funksie **true** terugstuur. Anders, moet die funksie **false** terugstuur. Aanvaar dat die aanvanklike funksieroep om die hele tou te deursoek soos volg is:*

```
myQ.binarySearch(myQ.begin(), 0, myQ.getNumItems(), myValue);
```

3.6 Do you think binary search is a good choice of a search algorithm for linked list-like structures? (2)

Explain your answer. /

Dink jy 'n binêre soektog is 'n goeie keuse vir 'n soekalgoritme vir geskakelde lys-tipe strukture. Verduidelik jou antwoord.

- 3.7 Bubble sort is a sorting algorithm that sweeps through a list repeatedly, swapping adjacent elements that are out of order, until no swaps are made anymore. A simplified version in which `list.length` number of sweeps are always made is defined as follows: / (7)

Borrelsorteer is 'n sorteer algoritme wat herhaaldelik deur 'n lys beweeg. Met elke beweging word buur-elemente wat uit plek uit is geruil, totdat geen ruilings meer gemaak word nie. 'n Vereenvoudigde weergawe, wat altyd `list.length` kere oor die lys beweeg, word as volg gedefinieer:

```
given a list
for sweep from 0 to list.length - 1
  for i from 0 to list.length - 2
    if list[i] > list[i+1]
      swap list[i] with list[i+1]
```

Implement the bubble sort algorithm using recursion. No loop structures (`for`, `while` or `do ... while`) are allowed. You will have to make use of indirect recursion (where two or more functions are involved) to achieve the same results as in the nested loops. You should also provide a simple function that clients can call without working knowledge of other functions. This function will then call one of the recursive functions with appropriate parameters to start the sorting. The function should have the following signature: /

Implementeer borrelsorteer deur van rekursie gebruik te maak. Geen lus-strukture (`for`, `while` of `do ... while`) mag gebruik word nie. Jy sal van indirekte rekursie (waar twee of meer funksies betrokke is) gebruik moet maak om dieselfde effek as die geneste lusse te verkry. Jy moet ook 'n eenvoudige funksie verskaf wat kliënte kan roep sonder enige kennis van die ander funksies. Hierdie funksie sal dan die sorteer algoritme begin deur 'n rekursiewe funksie te roep met gepaste parameters. Die funksie moet die volgende prototipe hê:

```
void bubblesort(int list [], int length);
```